**SOKOINE UNIVERSITY OF AGRICULTURE**

**School of Engineering and Technology**
**Department of Agricultural Engineering**

---

# AGE 219: Basics of Computer Programming

## Lecture Compendium 3: Control Flow — The Logic of Decision Making in Agricultural Systems

---

**Prepared by:**

Dr. Kadeghe Fue, PhD, P.Eng (T)

**Target Audience:** Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
**Primary Tools:** Python 3.x and Google Colaboratory
**Location:** Morogoro, Tanzania

*This document explores the fundamental mechanisms that allow computers to "think" and react. In this module, we transition from linear calculations to autonomous logic, enabling the creation of smart agricultural controllers and decision-support systems.*

# Contents


**1 Introduction: The Engineering Need for Intelligence** **2**
1.1 Linear vs. Conditional Execution . . . 2

**2 The Foundation of Logic: Boolean Algebra for Engineers** **2**
2.1 Comparison Operators: The Interrogators . . . 2
2.2 Truth Tables: Logical Gateways . . . 2
2.2.1 1. The `and` Operator (Intersection) . . . 3
2.2.2 2. The `or` Operator (Union) . . . 3

**3 Single-Branch Decisions: The `if` Statement** **3**
3.1 The Indentation Mandate . . . 3

**4 Binary Decisions: The `if-else` Structure** **3**

**5 Multi-State Logic: The `elif` Chain** **4**

**6 Complex Logic and Nested Conditionals** **4**

**7 Case Study 1: Comprehensive Frost Protection Logic** **4**

**8 Case Study 2: NPK Fertilizer Recommendation Engine** **5**

**9 Debugging Logical Errors in Control Flow** **5**
9.1 1. The "Exclusion" Error . . . 5
9.2 2. Comparison with Floats . . . 6

**10 Conclusion** **6**

**11 Glossary of Terms** **7**

**12 Extended Self-Assessment Exercises** **8**
12.1 Exercise 1: Autonomous Tractor Safety Interlock . . . 8
12.2 Exercise 2: Grain Bin Aeration Controller . . . 8
12.3 Exercise 3: Irrigation Pumping Tariff Logic . . . 8
12.4 Exercise 4: Complex Multi-Crop Grading . . . 8

# 1 Introduction: The Engineering Need for Intelligence

In our previous lectures, we treated the computer as a highly efficient calculator. We stored static values in variables and performed linear mathematical operations. While this is useful for basic tasks, real-world agricultural engineering problems are rarely linear. They are **conditional**.

Consider a modern, energy-efficient greenhouse. If we program it to simply turn on the ventilation at 8:00 AM every day, we are ignoring the reality of weather. An *intelligent* engineering system must be capable of observing its environment and branching its path of execution based on specific criteria.

## 1.1 Linear vs. Conditional Execution

- **Linear Execution:** The computer runs Line 1, then Line 2, then Line 3, regardless of the input. (e.g., A script that always converts 50 Acres to Hectares).
- **Conditional Execution:** The computer evaluates a state. Depending on whether that state is `True` or `False`, it chooses different sets of instructions. This is called **Control Flow**.

Control Flow is the fundamental building block of **Automation**. Without it, there would be no autonomous tractors, no smart irrigation, and no automated grain sorting.

# 2 The Foundation of Logic: Boolean Algebra for Engineers

To implement decision-making, we must return to the concept of the **Boolean** data type. In digital logic, everything boils down to 1s and 0s, or `True` and `False`. In Python, these are the only two values a condition can evaluate to.

## 2.1 Comparison Operators: The Interrogators

Comparison operators are the symbols we use to "ask the computer a question" about the data.

- == (Equal to): Tests for identical values.
- != (Not equal to): Tests if values are different.
- < (Less than): Tests magnitude.
- > (Greater than): Tests magnitude.
- <= (Less than or equal to).
- >= (Greater than or equal to).

```
# Example: Irrigation Threshold
threshold_psi = 40.0
current_psi = 38.5

# This expression evaluates to a Boolean
is_low_pressure = current_psi < threshold_psi
print(is_low_pressure) # Output: True
```

## 2.2 Truth Tables: Logical Gateways

As engineers, you may be familiar with Truth Tables from electronics. In Python, we use **Logical Operators** to combine multiple conditions.

### 2.2.1 1. The `and` Operator (Intersection)

The result is `True` **ONLY** if both conditions are True.

| Condition A | Condition B | Result (A `and` B) |
|---|---|---|
| False | False | False |
| False | True | False |
| True | False | False |
| **True** | **True** | **True** |

### 2.2.2 2. The `or` Operator (Union)

The result is `True` if **at least one** condition is True.

| Condition A | Condition B | Result (A `or` B) |
|---|---|---|
| False | False | False |
| False | True | True |
| True | False | True |
| **True** | **True** | **True** |

# 3 Single-Branch Decisions: The `if` Statement

The `if` statement is the most basic control structure. It tells Python: "Execute this block of code **only if** the condition evaluates to `True`. Otherwise, skip it."

## 3.1 The Indentation Mandate

Python is unique because it uses indentation to define the "scope" of a block. In most languages, curly braces `{}` are used. In Python, a colon (`:`) at the end of the `if` line followed by a 4-space indent on the next lines tells the computer which commands belong to the decision.

```
temperature_c = 35.0

if temperature_c > 32.0:
    # Everything indented is part of the 'True' branch
    print("ALERT: Ambient temperature exceeding safety limits!")
    print("Activating secondary cooling system.")

print("Monitoring continue...") # This line runs regardless of the temperature
```

# 4 Binary Decisions: The `if-else` Structure

Often, an engineering system has two distinct states. If the condition is met, do Action A. If it is **not** met, do Action B. This is handled by the `else` keyword.

```
soil_ph = 6.2

if soil_ph < 5.5:
    action = "Apply Lime to raise pH"
else:
```

```
    action = "Maintain current soil management"

print(f"Recommended Action: {action}")
```

Note that the `else` block never has its own condition. It is the "catch-all" for every scenario where the `if` condition is `False`.

## 5 Multi-State Logic: The `elif` Chain

In agricultural engineering, many systems have multiple operating modes. For example, a solar-powered irrigation pump might operate at 100% capacity in full sun, 50% in cloud, and 0% at night. We handle these using `elif` (short for **else if**).

```
solar_irradiance_wm2 = 450.0

if solar_irradiance_wm2 > 800:
    pump_speed = 100
elif solar_irradiance_wm2 > 300:
    pump_speed = 50
elif solar_irradiance_wm2 > 50:
    pump_speed = 10
else:
    pump_speed = 0

print(f"Inverter Output: {pump_speed}% power.")
```

**Execution Rule:** Python checks `elif` chains from top to bottom. As soon as it finds one `True` condition, it runs that block and **exits the entire chain**. It will not check the conditions below it. Therefore, order matters!

## 6 Complex Logic and Nested Conditionals

Sometimes, a decision depends on a hierarchy. For instance, in a post-harvest facility, we might only check the moisture content of grain *after* we have verified that the weighing scale is calibrated.

```
is_scale_calibrated = True
grain_moisture = 14.5

if is_scale_calibrated:
    if grain_moisture < 13.0:
        status = "Ready for Storage"
    else:
        status = "Requires further drying"
else:
    status = "ERROR: Calibration Required"

print(f"System Status: {status}")
```

## 7 Case Study 1: Comprehensive Frost Protection Logic

**Engineering Context:** Frost can destroy an entire orchard's yield in a single night. Engineers design automated sprinkler systems that turn on when the air temperature drops to a critical level, but *only* if the wind speed is low (high wind makes the water cool the trees even faster via evaporation).

```python
# --- Inputs ---
current_temp_c = -1.5
wind_speed_ms = 2.0
sprinkler_system_fault = False

# --- Decision Tree ---
if sprinkler_system_fault:
    msg = "SYSTEM ERROR: Sprinkler fault detected. Contact maintenance."
elif current_temp_c <= 0.0:
    if wind_speed_ms < 5.0:
        msg = "ACTIVE: Commencing frost protection irrigation."
    else:
        msg = "WARNING: Temperature low but wind too high. Risk of evaporative cooling."
else:
    msg = "STATUS: Temperature safe. Monitoring..."

print(f"Frost Monitor: {msg}")
```

# 8 Case Study 2: NPK Fertilizer Recommendation Engine

**Engineering Context:** Precision agriculture involves adjusting fertilization based on soil tests. Let's build a logic engine for Nitrogen (N) requirements.

```python
# --- Inputs ---
soil_nitrogen_ppm = 15.0
target_crop = "Maize" # Could be "Maize", "Rice", or "Beans"

# --- Logic ---
if target_crop == "Maize":
    if soil_nitrogen_ppm < 10:
        recommendation = "Apply 150 kg/ha N"
    elif soil_nitrogen_ppm < 30:
        recommendation = "Apply 100 kg/ha N"
    else:
        recommendation = "N level sufficient"
elif target_crop == "Beans":
    if soil_nitrogen_ppm < 5:
        recommendation = "Apply 20 kg/ha N"
    else:
        recommendation = "N level sufficient"
else:
    recommendation = "Crop not in database"

print(f"Fertilizer Plan: {recommendation}")
```

# 9 Debugging Logical Errors in Control Flow

Logical errors are the most difficult to fix because the program "runs" but the "result" is wrong. In irrigation or machinery control, a logical error can cause expensive damage.

## 9.1 1. The "Exclusion" Error

Consider this code meant to categorize soil moisture:

```python
moisture = 45
if moisture > 20:
    print("Normal")
elif moisture > 40:
    print("Saturated")
```

**The Error:** Since 45 is greater than 20, the computer prints "Normal" and stops. It never reaches the "Saturated" check.
**The Fix:** Always put the **most restrictive** or **highest values** at the top of an `elif` chain.

### 9.2 2. Comparison with Floats

Because of how computers store binary numbers, `0.1 + 0.2` is actually `0.30000000000000004`.

```
val = 0.1 + 0.2
if val == 0.3:
    print("Equal")
else:
    print("Not Equal") # THIS WILL RUN
```

**The Fix:** Use a tolerance (epsilon). `if abs(val - 0.3) < 0.0001:`

## 10 Conclusion

Control flow turns Python from a calculation tool into a **decision-making system**. By combining sensors (inputs), variables (storage), and conditional logic (if-elif-else), you can build the "brains" for any agricultural technology. In the next lecture, we will learn how to repeat these decisions using **Loops**.

## 11 Glossary of Terms

- **Boolean Expression:** Any code that results in either `True` or `False`.
- **Branching:** The process where a program chooses between two or more different paths of execution.
- **Control Flow:** The order in which the program's code executes.
- **Epsilon ($\epsilon$):** A very small value used in engineering to compare floating-point numbers for "near equality."
- **Indentation:** The spaces (typically 4) used to define code blocks in Python.
- **Logical Operators:** `and`, `or`, and `not`—used to combine or invert Boolean conditions.
- **Nested Conditional:** A decision structure placed inside another decision structure.
- **Truth Table:** A mathematical table used in logic to determine the output based on all possible combinations of inputs.

# 12 Extended Self-Assessment Exercises

## 12.1 Exercise 1: Autonomous Tractor Safety Interlock

A tractor has three sensors: `is_person_in_seat` (Boolean), `is_obstacle_ahead` (Boolean), and `is_engine_overheating` (Boolean). **Task:** Write a script that sets a variable `can_move` to `True` only if a person is in the seat **AND** there is no obstacle **AND** the engine is not overheating. Otherwise, set it to `False` and print the specific reason for the stop.

## 12.2 Exercise 2: Grain Bin Aeration Controller

Grain in a bin will spoil if it is too moist. An aeration fan should only turn on if:

1. The grain moisture is above 14%
2. **AND** the outside air humidity is below 70% (to ensure the air can actually dry the grain).

**Task:** Write a script that takes `grain_moisture` and `air_humidity` as inputs and prints whether the fan is ”ON” or ”OFF”.

## 12.3 Exercise 3: Irrigation Pumping Tariff Logic

Electricity costs in Tanzania vary by time.

- Peak (08:00 to 22:00): 450 TZS/kWh
- Off-Peak (22:01 to 07:59): 250 TZS/kWh

**Task:** Write a script that takes an integer `hour` (0-23) and prints the current tariff. Use `if-else`.

## 12.4 Exercise 4: Complex Multi-Crop Grading

Create a script that grades either ”Rice” or ”Maize”.

- Rice Grade A: Moisture < 13% and Broken < 5%
- Maize Grade A: Moisture < 14% and Broken < 3%

Ask the user for the crop type, moisture, and broken percentage. Output the final grade or ”Reject”.

---

*End of Lecture 3 Compendium*